Relatório Final — Otimização de Pipeline MLOps para Manutenção Preditiva Industrial

Disciplina: Engenharia de Dados e MLOps — Aula 09 **Cenário:** Fábrica TechIndustries — manutenção preditiva de máquina crítica **Autor:** Matheus Silva da Cruz

---

1. Contexto

Uma parada não planejada (falha não detectada, **Falso Negativo**) custa **R$ 15.000,00** por ocorrência, enquanto uma manutenção preventiva disparada por engano (**Falso Positivo**) custa apenas **R$ 500,00**. O objetivo é escolher o modelo que proporcione o **menor custo financeiro total** nos três últimos meses do ano (out–dez/2025).

As melhorias implementadas no arquivo `app_manutencao_otimizado.py` foram:

- **Tarefa 2 — Engenharia de Features Avançada:** novas variáveis temporais de tendência de degradação.
- **Tarefa 3 — Balanceamento com SMOTE:** reamostragem sintética da classe minoritária, aplicada **somente no treino**.
- **Tarefa 4 — Análise de Custo Financeiro:** matriz de custo de negócio e tabela comparativa.

---

2. Tarefa 2 — Engenharia de Features Avançada

Foram adicionadas as seguintes variáveis, que capturam a *velocidade* e os *extremos* recentes do comportamento dos sensores:

Feature	Descrição
`taxa_var_vibracao`	Diferença percentual (`pct_change`) da vibração entre horas consecutivas.
`taxa_var_pressao`	Diferença percentual (`pct_change`) da pressão entre horas consecutivas.
`temperatura_max_12h`, `temperatura_min_12h`	Máximo e mínimo da temperatura em janela móvel de 12 h.
`vibracao_max_12h`, `vibracao_min_12h`	Máximo e mínimo da vibração em janela móvel de 12 h.
`temp_x_vibracao`	Feature de interação `temperatura * vibracao` (a falha ocorre quando ambas estão altas).

---

3. Pergunta 1 — Impacto das novas variáveis no *Recall* dos modelos de Gradient Boosting

Comparação do **Recall** (sensibilidade) dos três modelos de Gradient Boosting entre a versão **base** (apenas ponderação de classe: `scale_pos_weight`) e a versão **otimizada** (novas features temporais **+ SMOTE**):

Modelo	Recall — Base	Recall — Otimizado	Variação
XGBoost	2,00%	**8,00%**	▲ +6,00 p.p. (4×)
CatBoost	8,00%	**16,00%**	▲ +8,00 p.p. (2×)

Modelo	Recall — Base	Recall — Otimizado	Variação
LightGBM	20,00%	20,00%	= estável

Resposta: As novas variáveis temporais, combinadas com o SMOTE, **aumentaram o Recall** dos modelos de Gradient Boosting que antes estavam quase "cegos" para falhas. O **XGBoost** quadruplicou seu Recall (de 2% para 8%) e o **CatBoost** dobrou (de 8% para 16%), passando a detectar mais falhas reais. O **LightGBM** manteve 20%. O ganho de Recall vem da combinação de dois fatores: as features de interação/tendência (especialmente `temp_x_vibracao`, que reproduz a própria regra de falha) oferecem um sinal mais forte, e o SMOTE equilibra as classes no treino, reduzindo o viés dos modelos para a classe majoritária "Normal". Ainda assim, o Recall dos modelos de boosting permanece moderado porque boa parte das falhas é composta de **ruído aleatório** (2% das amostras) — eventos intrinsecamente imprevisíveis a partir dos sensores.

> *Observação:* os valores exatos podem variar levemente conforme a versão das bibliotecas, mas a tendência de melhoria do Recall do XGBoost e do CatBoost é consistente.

---

4. Tarefa 3 — Balanceamento com SMOTE

- A classe de falhas é fortemente minoritária no treino: **6.400 normais × 129 falhas**.
- Após o SMOTE (aplicado **apenas em** `X_treino`/`y_treino`): **6.400 × 6.400** (classes equilibradas).
- O SMOTE **não foi aplicado** na base de teste, evitando *data leakage* e preservando a validade da avaliação.
- Com o treino já equilibrado, os parâmetros de ponderação (`scale_pos_weight` / `class_weight`) foram removidos.

---

5. Tarefa 4 — Tabela de Custo Financeiro Industrial

Fórmula de negócio:

> **Custo Total (R$) = (Falsos Negativos × R$ 15.000) + (Falsos Positivos × R$ 500)**

Resultado na avaliação dos 3 últimos meses (ordenado do menor para o maior custo):

Modelo	Recall	Precisão	Alarmes Falsos (FP)	Falhas Perdidas (FN)	Custo Financeiro Total (R$)
CatBoost	16,00%	3,52%	219	42	**R$ 739.500,00**
XGBoost	8,00%	2,78%	140	46	R$ 760.000,00
LightGBM	20,00%	2,92%	332	40	R$ 766.000,00
Random Forest	26,00%	2,87%	440	37	R$ 775.000,00
Regressão Logística	46,00%	2,51%	894	27	R$ 852.000,00

O painel visual com as matrizes de confusão está em `matriz_comparativa_otimizada.png`.

---

6. Recomendação — Qual modelo promover para produção

Modelo recomendado: CatBoost — menor custo financeiro total: **R$ 739.500,00**.

Fundamentação sob a ótica de negócios:

- O critério de decisão é **financeiro**, não apenas o Recall. Embora a Regressão Logística tenha o maior Recall (46%), ela dispara **894 alarmes falsos**, e esse excesso de manutenções desnecessárias a torna a opção **mais cara** (R$ 852.000,00).
- O **CatBoost** atinge o melhor equilíbrio entre capturar falhas (Recall 16%) e **não poluir a operação com alarmes falsos** (apenas 219 FP), resultando no **menor custo total**.
- Como o custo de uma Falha Perdida (R$ 15.000) é 30× o de um Alarme Falso (R$ 500), o modelo precisa reduzir FN sem explodir os FP — e o CatBoost é o que melhor atende a esse trade-off nos dados de teste.

Ressalvas de MLOps

- Todos os modelos operam em **custo elevado** porque a maioria das falhas deste dataset sintético é **ruído aleatório** (imprevisível pelos sensores). Em um cenário real, convém revisar a geração/qualidade dos dados e as features disponíveis.
- Recomenda-se, em produção, **otimizar o limiar de decisão** (threshold) de cada modelo diretamente pela **minimização do custo financeiro** (e não pelo padrão 0,5), pois isso tende a reduzir ainda mais o custo total.
- Monitorar *data drift* e re-treinar periodicamente, mantendo o SMOTE restrito ao treino.

---

7. Como reproduzir

```
python -m venv venv
# Windows:  .\venv\Scripts\Activate.ps1
# Linux/Mac: source venv/bin/activate
pip install -r requirements.txt
pip install imbalanced-learn
python app_manutencao_otimizado.py
```

Artefatos gerados: `telemetria_maquina_2025.csv`, `dados_teste_ultimos_3_meses.csv` e `matriz_comparativa_otimizada.png`.